Course: CS420 – Artificial Intelligence

Midterm examination review – Lectures 0 to 3

PRACTICE MIDTERM EXAMINATION – TEST 1
SOLUTION

Duration: 120 minutes · Closed book · Calculator allowed

Unless a question says otherwise, ties are broken in alphabetical order (left-to-right in trees), and successors are generated in alphabetical order.

Question 1. *The two-jug problem.* You have a 4-litre jug and a 3-litre jug, both initially empty, and a tap with unlimited water. You may fill a jug from the tap, empty a jug onto the ground, or pour one jug into the other until the source is empty or the destination is full. The goal is to have *exactly 2 litres in the 4-litre jug.*

(a) Consider a state as a tuple of values. Which values should be included in the tuple?

(x, y): litres in the 4-L jug ($x \in \{0, \ldots, 4\}$) and in the 3-L jug ($y \in \{0, \ldots, 3\}$).

(b) Define the initial state and the goal state(s).

Initial $(0, 0)$. Goal: any state with $x = 2$, i.e. $(2, y)$ for $y = 0..3$.

(c) Define the action function (preconditions and effects of every action).

Fill4: $x < 4 \Rightarrow (4, y)$; Fill3: $y < 3 \Rightarrow (x, 3)$; Empty4: $x > 0 \Rightarrow (0, y)$; Empty3: $y > 0 \Rightarrow (x, 0)$; Pour4→3: $x > 0, y < 3,\ t = \min(x, 3 - y) \Rightarrow (x - t,\ y + t)$; Pour3→4: $y > 0, x < 4,\ t = \min(y, 4 - x) \Rightarrow (x + t,\ y - t)$.

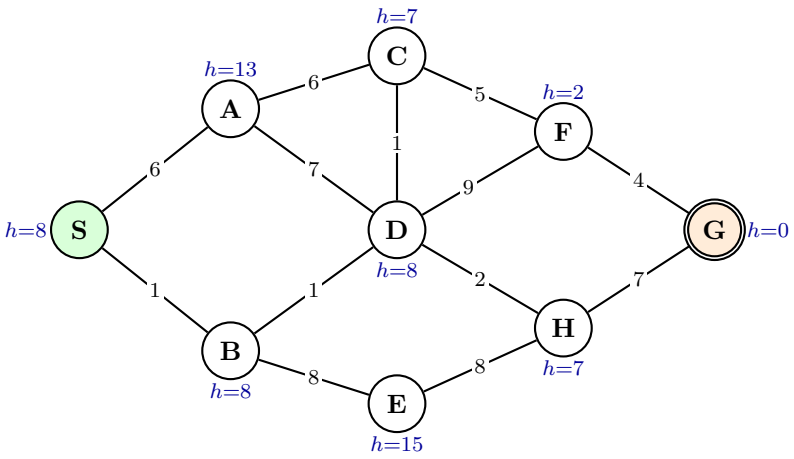
(d) What is the action cost?

Each action costs 1 (path cost = number of actions).

(e) What is the maximum number of states in the state space? How many are reachable from the initial state? Give a shortest solution.

$5 \times 4 =$ **20** tuples. Reachable from $(0, 0)$: **14** (every reachable state has at least one jug empty or full). Shortest solution (6 actions): (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3).

Question 2. Consider the undirected graph below; S is the initial state and G the goal state. Step costs are written on the edges and the heuristic h next to the vertices.



For each search strategy, (a) list, in order, the states **expanded**, (b) give the path found and (c) give the final content of the frontier (a state is expanded when it is removed from the frontier). Conventions: BFS, DFS and greedy best-first search test a state for the goal when it is *generated* (so the goal is not expanded); UCS and A* test it when it is *removed* from the frontier (graph search; a cheaper path to a frontier state replaces its entry). DFS avoids loops by remembering the states on the current path only. The path returned will not be accepted if the list of expanded states is wrong.

Algorithm	**List of expanded states**	**Path returned**	**Final frontier**
Breadth-first search	S A B C D E F	S A C F G (cost 21)	{ H }
Uniform-cost search	S B D C H A F E G	S B D H G (cost 11)	{ }
Depth-first search	S A C D B E H	S A C D B E H G (cost 37)	(not asked)
Greedy best-first search	S B D F	S B D F G (cost 15)	{ C, H, A, E }
Graph-search A*	S B D C F H G	S B D H G (cost 11)	{ A, E }

BFS (queue after each expansion):

Step	Expanded	Queue (front → back) after the expansion
1	S	A, B
2	A	B, C, D
3	B	C, D, E
4	C	D, E, F
5	D	E, F, H
6	E	F, H
7	F	generates goal **G** ⇒ stop; frontier: H

DFS (stack after each expansion):

Step	Expanded	Stack (top → bottom) after the expansion
1	S	A, B
2	A	C, D, B
3	C	D, F, D, B
4	D	B, F, H, F, D, B
5	B	E, F, H, F, D, B
6	E	H, F, H, F, D, B
7	H	generates goal **G** ⇒ stop; frontier: F, H, F, D, B

UCS:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g(n)$)
1	S_0	B_1, A_6
2	B_1	D_2, A_6, E_9
3	D_2	C_3, H_4, A_6, E_9, F_{11}
4	C_3	H_4, A_6, F_8, E_9
5	H_4	A_6, F_8, E_9, G_{11}
6	A_6	F_8, E_9, G_{11}
7	F_8	E_9, G_{11}
8	E_9	G_{11}
9	G_{11}	*goal removed from the frontier – stop*

Greedy best-first:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $h(n)$)
1	S_8	B_8, A_{13}
2	B_8	D_8, A_{13}, E_{15}
3	D_8	F_2, C_7, H_7, A_{13}, E_{15}
4	F_2	generates goal **G** ⇒ stop; frontier: C_7, H_7, A_{13}, E_{15}

A*:

Step	Expanded	Frontier after the expansion, in priority order (subscript = $g + h = f$)
1	$S_{0+8=8}$	$B_{1+8=9}$, $A_{6+13=19}$
2	$B_{1+8=9}$	$D_{2+8=10}$, $A_{6+13=19}$, $E_{9+15=24}$
3	$D_{2+8=10}$	$C_{3+7=10}$, $H_{4+7=11}$, $F_{11+2=13}$, $A_{6+13=19}$, $E_{9+15=24}$
4	$C_{3+7=10}$	$F_{8+2=10}$, $H_{4+7=11}$, $A_{6+13=19}$, $E_{9+15=24}$
5	$F_{8+2=10}$	$H_{4+7=11}$, $G_{12+0=12}$, $A_{6+13=19}$, $E_{9+15=24}$
6	$H_{4+7=11}$	$G_{11+0=11}$, $A_{6+13=19}$, $E_{9+15=24}$
7	$G_{11+0=11}$	*goal removed from the frontier – stop*

(d) Is h admissible? Justify.

True costs h^*: S= 11, A= 15, B= 10, C= 9, D= 9, E= 15, F= 4, H= 7, G= 0. $h(n) \leq h^*(n)$ for every n ⇒ **admissible**.

(e) Is h consistent? Justify.

$h(u) \leq c(u, v) + h(v)$ holds on every edge in both directions (tight: $h(A) = c(A, C) + h(C)$, $h(C) = c(C, F) + h(F)$, $h(D) = c(D, C) + h(C)$, $h(E) = c(E, H) + h(H)$, $h(H) = c(H, G) + h(G)$) ⇒ **consistent**.

(f) Which of the five strategies returned an optimal path here, and which of them are guaranteed to do so?

Optimal cost 11. UCS and A* found it; BFS (cost 21), DFS (37) and GBFS (15) did not. UCS (positive costs) and A* with an admissible/consistent heuristic are guaranteed optimal; BFS only when all step costs are equal; DFS and GBFS never.

Question 3. Consider the 8-puzzle. Apply *steepest-descent hill climbing* with the Manhattan-distance heuristic h to the following pair of initial and goal states.

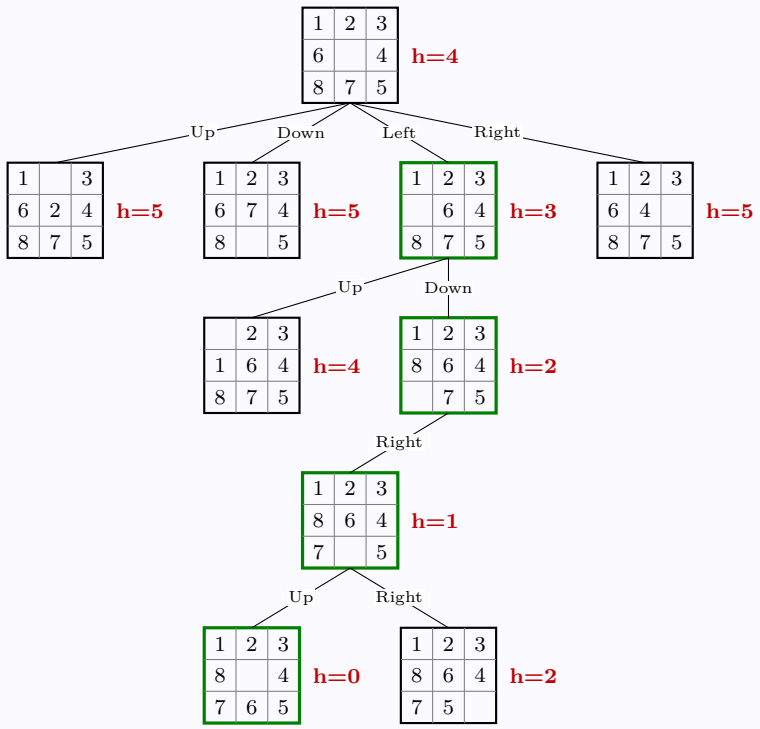
Initial state

1	2	3
6	–	4
8	7	5

Goal state

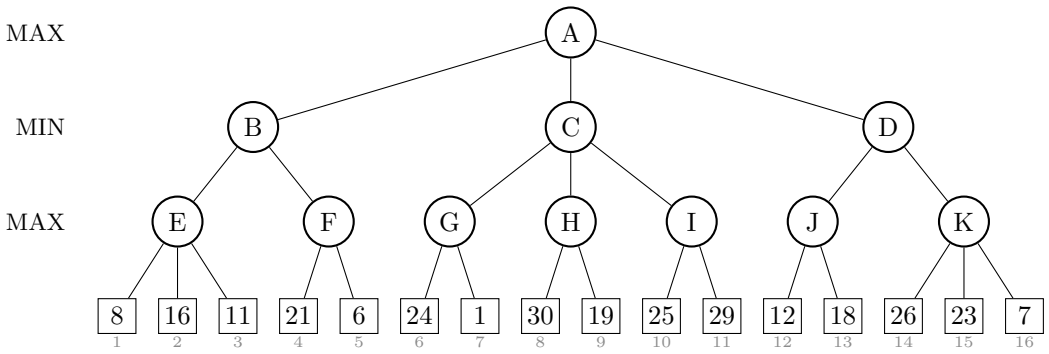
1	2	3
8	–	4
7	6	5

Your work should: draw the search tree including all successors of every expanded state (moves of the blank in the order Up, Down, Left, Right; do not regenerate the parent state), write the heuristic value of every node, and mark the path found.



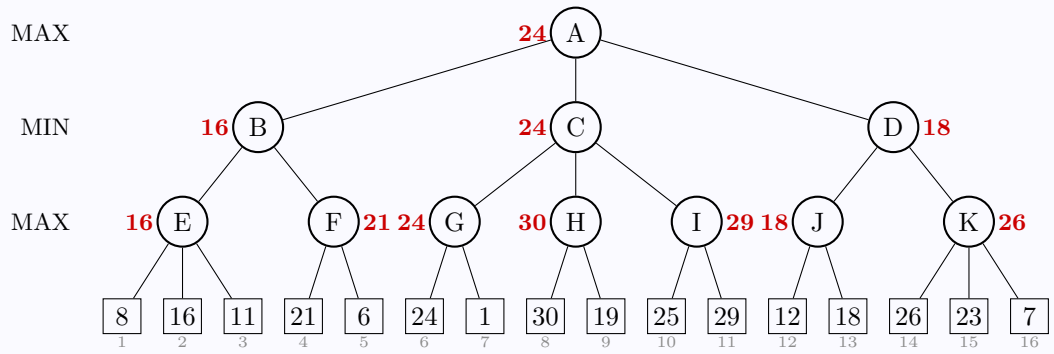
Each level keeps the successor with the smallest h (green). h decreases 4→3→2→1→0 and the goal is reached after 4 moves: Left, Down, Right, Up.

Question 4. Consider the following game tree. The root is a MAX node and the search always visits children left to right (small grey numbers are leaf indices).



(a) Compute the final backed-up value of every node using the minimax algorithm (no alpha-beta pruning at this step). Which move should MAX make at the root?

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
Minimax	24	16	24	18	16	21	24	30	29	18	26

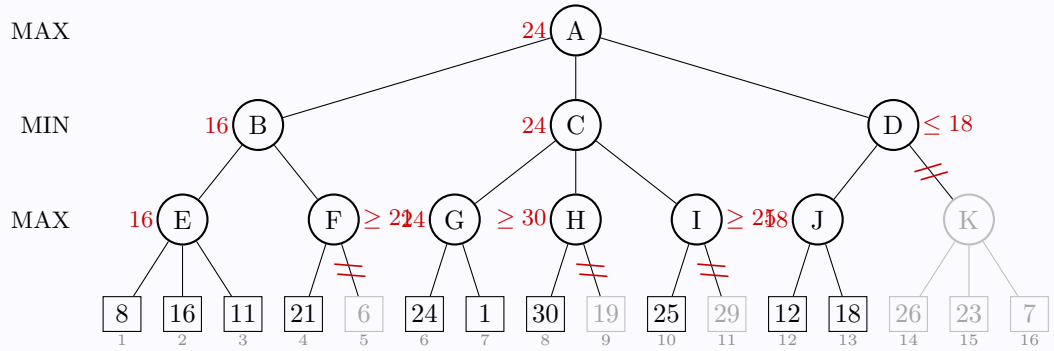


Root value = 24; MAX moves to **C**.

(b) Run alpha-beta pruning (children visited left to right). Give the value computed at every node (mark **X** if the node is never visited), cross out the pruned branches on the tree and briefly explain each case.

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**
α-β	24	16	24	≤ 18	16	≥ 21	24	≥ 30	≥ 25	18	**X**

($\leq v$ / $\geq v$: the node was cut off, so only this bound is known; writing the returned number v is also accepted.)



α and β at every visited node:

Node	Type	On entry	When it returns	Value returned
A	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 24,\ \beta = +\infty$	24
B	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 16$	16
E	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 16,\ \beta = +\infty$	16
F	MAX	$\alpha = -\infty,\ \beta = 16$	$\alpha = 21,\ \beta = 16$	≥ 21
C	MIN	$\alpha = 16,\ \beta = +\infty$	$\alpha = 16,\ \beta = 24$	24
G	MAX	$\alpha = 16,\ \beta = +\infty$	$\alpha = 24,\ \beta = +\infty$	24
H	MAX	$\alpha = 16,\ \beta = 24$	$\alpha = 30,\ \beta = 24$	≥ 30
I	MAX	$\alpha = 16,\ \beta = 24$	$\alpha = 25,\ \beta = 24$	≥ 25
D	MIN	$\alpha = 24,\ \beta = +\infty$	$\alpha = 24,\ \beta = 18$	≤ 18
J	MAX	$\alpha = 24,\ \beta = +\infty$	$\alpha = 24,\ \beta = +\infty$	18

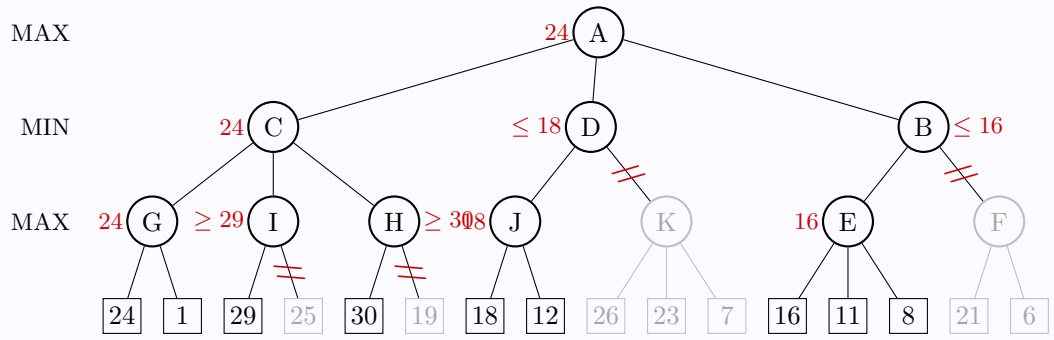
- At MAX node **F**, after leaf #4: $v = 21 \geq \beta = 16 \Rightarrow$ prune leaf #5. (MIN at B already has an alternative worth $\beta = 16$, so it will never let play reach F.)
- At MAX node **H**, after leaf #8: $v = 30 \geq \beta = 24 \Rightarrow$ prune leaf #9. (MIN at C already has an alternative worth $\beta = 24$, so it will never let play reach H.)

- At MAX node **I**, after leaf #10: $v = 25 \geq \beta = 24 \Rightarrow$ prune leaf #11. (MIN at C already has an alternative worth $\beta = 24$, so it will never let play reach I.)
- At MIN node **D**, after child J: $v = 18 \leq \alpha = 24 \Rightarrow$ prune K (leaves #14–#16). (MAX at A already has an alternative worth $\alpha = 24$, so it will never let play reach D.)

Leaves evaluated: 10 of 16.

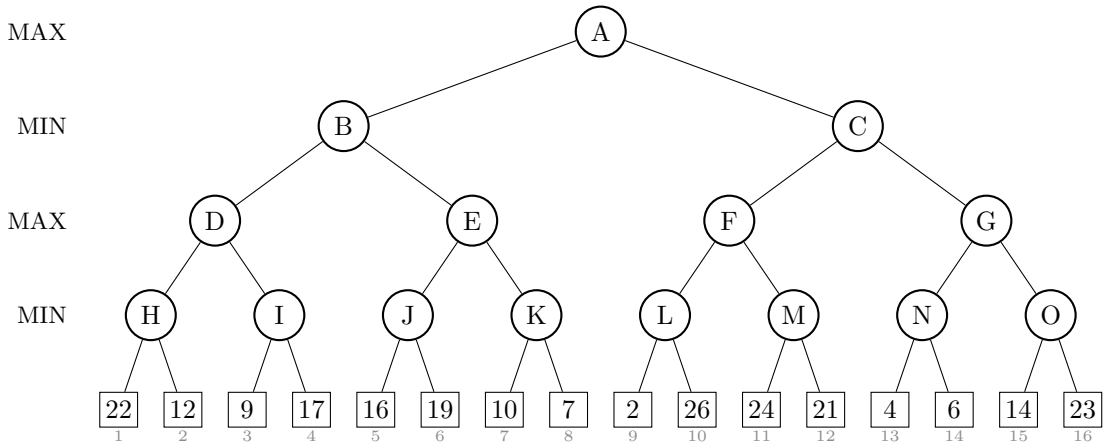
(c) Using the minimax values from (a), and without performing any alpha-beta calculation, rotate the children of each node at every level so that alpha-beta prunes as much as possible. Draw the new tree (keep the node letters).

Best child first for the player to move (MAX: largest value first, MIN: smallest first):



With this ordering alpha-beta evaluates only **9** leaves (instead of 10).

Question 5. Consider the following 4-ply game tree (MAX at the root).

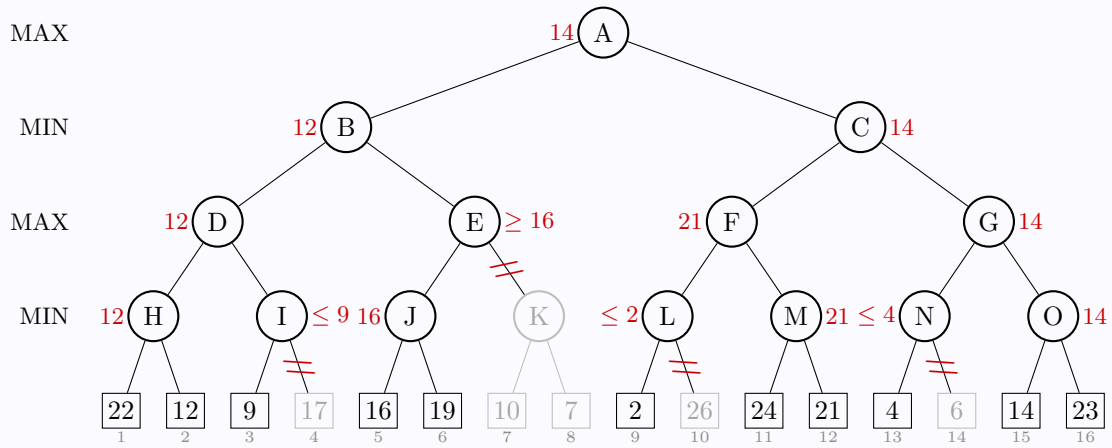


(a) Note the minimax value of every node (no pruning).

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**	**H**	**I**	**J**	**K**	**L**	**M**	**N**	**O**
Minimax	14	12	14	12	16	21	14	12	9	16	7	2	21	4	14

Root value = 14; MAX moves to **C**.

(b) Show where alpha-beta pruning occurs (left to right) by crossing out the corresponding branches, and briefly explain each case.



Node	Type	On entry	When it returns	Value returned
A	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 14,\ \beta = +\infty$	14
B	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 12$	12
D	MAX	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = 12,\ \beta = +\infty$	12
H	MIN	$\alpha = -\infty,\ \beta = +\infty$	$\alpha = -\infty,\ \beta = 12$	12
I	MIN	$\alpha = 12,\ \beta = +\infty$	$\alpha = 12,\ \beta = 9$	≤ 9
E	MAX	$\alpha = -\infty,\ \beta = 12$	$\alpha = 16,\ \beta = 12$	≥ 16
J	MIN	$\alpha = -\infty,\ \beta = 12$	$\alpha = -\infty,\ \beta = 12$	16
C	MIN	$\alpha = 12,\ \beta = +\infty$	$\alpha = 12,\ \beta = 14$	14
F	MAX	$\alpha = 12,\ \beta = +\infty$	$\alpha = 21,\ \beta = +\infty$	21
L	MIN	$\alpha = 12,\ \beta = +\infty$	$\alpha = 12,\ \beta = 2$	≤ 2
M	MIN	$\alpha = 12,\ \beta = +\infty$	$\alpha = 12,\ \beta = 21$	21
G	MAX	$\alpha = 12,\ \beta = 21$	$\alpha = 14,\ \beta = 21$	14
N	MIN	$\alpha = 12,\ \beta = 21$	$\alpha = 12,\ \beta = 4$	≤ 4
O	MIN	$\alpha = 12,\ \beta = 21$	$\alpha = 12,\ \beta = 14$	14

- At MIN node **I**, after leaf #3: $v = 9 \leq \alpha = 12 \Rightarrow$ prune leaf #4. (MAX at D already has an alternative worth $\alpha = 12$, so it will never let play reach I.)
- At MAX node **E**, after child J: $v = 16 \geq \beta = 12 \Rightarrow$ prune K (leaves #7–#8). (MIN at B already has an alternative worth $\beta = 12$, so it will never let play reach E.)
- At MIN node **L**, after leaf #9: $v = 2 \leq \alpha = 12 \Rightarrow$ prune leaf #10. (MAX at A already has an alternative worth $\alpha = 12$, so it will never let play reach L.) **Deep cut-off**: the bound comes from A, 3 levels above L.
- At MIN node **N**, after leaf #13: $v = 4 \leq \alpha = 12 \Rightarrow$ prune leaf #14. (MAX at A already has an alternative worth $\alpha = 12$, so it will never let play reach N.) **Deep cut-off**: the bound comes from A, 3 levels above N.

Leaves evaluated: 11 of 16; never evaluated: #4, #7, #8, #10, #14.
Deep cut-offs at L and N: the bound $\alpha = 12$ was set at the root A, several levels above, while their own parents had not found anything yet.

(c) Which of the cut-offs are *deep* cut-offs (the bound was set by an ancestor that is not the node's parent)?

L, N (bound from A); the others (I, E) use a bound set by the parent.

Question 6. Run Monte Carlo Tree Search from a tree that contains only the root S_0 ($t = 0$, $n = 0$), exactly as in the lecture walkthrough. Use UCB1$(S_i) = \bar{V}_i + C\sqrt{\ln N / n_i}$ with $C = 2$ (an unvisited node has UCB1 $= \infty$; ties go to the left-most node). A leaf that has never been visited is rolled out directly; otherwise all its children are added and the rollout starts at the first one. Every state has two actions: $S_0 \to S_1, S_2$; $S_1 \to S_3, S_4$; $S_2 \to S_5, S_6$; $S_3 \to S_7, S_8$; $S_4 \to S_9, S_{10}$; $S_5 \to S_{11}, S_{12}$. The rollouts of iterations 1–7 return, in order: 10, 7, 11, 4, 4, 3, 1.

(a) Perform the 7 iterations. Give the UCB1 values compared at every selection step and draw the final tree with t and n for every node.

Iteration 1.

- *Selection*: the root is the only node in the tree.
- *Expansion*: none — S_0 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from S_0 returns 10.
- *Backpropagation*: add 10 to t and 1 to n for S_0.

Statistics (t/n) after iteration 1: S_0: 10/1.

Iteration 2.

- *Selection*: the root is the only node in the tree.
- *Expansion*: S_0 was visited before ($n > 0$) and has no children ⇒ add S_1, S_2; roll out from the first one, S_1.
- *Simulation*: rollout from S_1 returns 7.
- *Backpropagation*: add 7 to t and 1 to n for $S_0 \to S_1$.

Statistics (t/n) after iteration 2: S_0: 17/2, S_1: 7/1, S_2: 0/0.

Iteration 3.

- *Selection* at S_0 ($N = 2$): S_1: $\frac{7}{1} + 2\sqrt{\frac{\ln 2}{1}} = 8.665$; S_2: ∞ (unvisited) ⇒ go to S_2.
- *Expansion*: none — S_2 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from S_2 returns 11.
- *Backpropagation*: add 11 to t and 1 to n for $S_0 \to S_2$.

Statistics (t/n) after iteration 3: S_0: 28/3, S_1: 7/1, S_2: 11/1.

Iteration 4.

- *Selection* at S_0 ($N = 3$): S_1: $\frac{7}{1} + 2\sqrt{\frac{\ln 3}{1}} = 9.096$; S_2: $\frac{11}{1} + 2\sqrt{\frac{\ln 3}{1}} = 13.096$ ⇒ go to S_2.
- *Expansion*: S_2 was visited before ($n > 0$) and has no children ⇒ add S_5, S_6; roll out from the first one, S_5.
- *Simulation*: rollout from S_5 returns 4.
- *Backpropagation*: add 4 to t and 1 to n for $S_0 \to S_2 \to S_5$.

Statistics (t/n) after iteration 4: S_0: 32/4, S_1: 7/1, S_2: 15/2, S_5: 4/1, S_6: 0/0.

Iteration 5.

- *Selection* at S_0 ($N = 4$): S_1: $\frac{7}{1} + 2\sqrt{\frac{\ln 4}{1}} = 9.355$; S_2: $\frac{15}{2} + 2\sqrt{\frac{\ln 4}{2}} = 9.165$ ⇒ go to S_1.
- *Expansion*: S_1 was visited before ($n > 0$) and has no children ⇒ add S_3, S_4; roll out from the first one, S_3.
- *Simulation*: rollout from S_3 returns 4.

- *Backpropagation*: add 4 to t and 1 to n for $S_0 \to S_1 \to S_3$.

Statistics (t/n) after iteration 5: S_0: 36/5, S_1: 11/2, S_3: 4/1, S_4: 0/0, S_2: 15/2, S_5: 4/1, S_6: 0/0.

Iteration 6.

- *Selection* at S_0 ($N = 5$): S_1: $\frac{11}{2} + 2\sqrt{\frac{\ln 5}{2}} = 7.294$; S_2: $\frac{15}{2} + 2\sqrt{\frac{\ln 5}{2}} = 9.294 \Rightarrow$ go to S_2.
- *Selection* at S_2 ($N = 2$): S_5: $\frac{4}{1} + 2\sqrt{\frac{\ln 2}{1}} = 5.665$; S_6: ∞ (unvisited) $\Rightarrow$ go to S_6.
- *Expansion*: none — S_6 has $n = 0$, so we roll out directly from it.
- *Simulation*: rollout from S_6 returns 3.
- *Backpropagation*: add 3 to t and 1 to n for $S_0 \to S_2 \to S_6$.

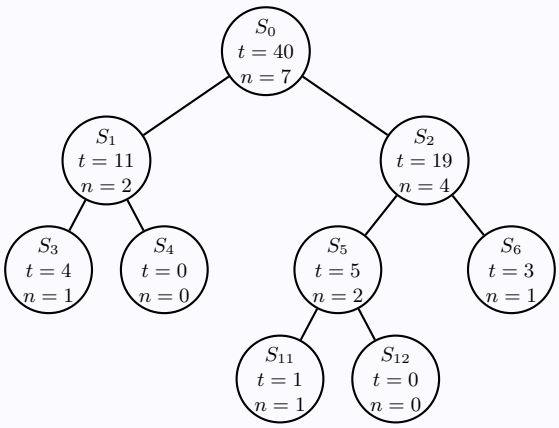
Statistics (t/n) after iteration 6: S_0: 39/6, S_1: 11/2, S_3: 4/1, S_4: 0/0, S_2: 18/3, S_5: 4/1, S_6: 3/1.

Iteration 7.

- *Selection* at S_0 ($N = 6$): S_1: $\frac{11}{2} + 2\sqrt{\frac{\ln 6}{2}} = 7.393$; S_2: $\frac{18}{3} + 2\sqrt{\frac{\ln 6}{3}} = 7.546 \Rightarrow$ go to S_2.
- *Selection* at S_2 ($N = 3$): S_5: $\frac{4}{1} + 2\sqrt{\frac{\ln 3}{1}} = 6.096$; S_6: $\frac{3}{1} + 2\sqrt{\frac{\ln 3}{1}} = 5.096 \Rightarrow$ go to S_5.
- *Expansion*: S_5 was visited before ($n > 0$) and has no children $\Rightarrow$ add S_{11}, S_{12}; roll out from the first one, S_{11}.
- *Simulation*: rollout from S_{11} returns 1.
- *Backpropagation*: add 1 to t and 1 to n for $S_0 \to S_2 \to S_5 \to S_{11}$.

Statistics (t/n) after iteration 7: S_0: 40/7, S_1: 11/2, S_3: 4/1, S_4: 0/0, S_2: 19/4, S_5: 5/2, S_{11}: 1/1, S_{12}: 0/0, S_6: 3/1.

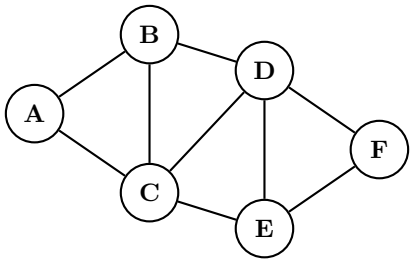
Final tree:



(b) Which action is recommended at the root by the *max child* rule, and which by the *robust child* rule?

Averages: S_1: $11/2 = 5.5$, S_2: $19/4 = 4.75 \Rightarrow$ max child S_1. Visits: S_1: 2, S_2: 4 $\Rightarrow$ robust child S_2. The rules disagree, so max-robust would continue the search.

Question 7. This problem is about map colouring. Each region must be coloured Red (R), Green (G) or Blue (B), and neighbouring regions must have different colours. The constraint graph is shown below. The following questions are mutually independent.



(a) Cross out all values that would be eliminated by *forward checking* after variable B has been assigned as shown.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	R G B	**R**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	~~R~~ G B	**R** ~~G B~~	~~R~~ G B	~~R~~ G B	R G B	R G B

Only the neighbours of B (A, C, D) lose R.

(b) A and B have been assigned as shown, but no constraint propagation has been done yet. **Cross out** all values that would be eliminated by *arc consistency* (AC-3).

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	**G**	R G B	R G B	R G B	R G B

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R** ~~G B~~	~~R~~ **G** ~~B~~	~~R G~~ B	R ~~G B~~	~~R~~ G ~~B~~	~~R G~~ B

Propagation continues through the graph (unlike forward checking): every arc $X_i \to X_j$ is revised until no domain changes. Here arc consistency even leaves a single value for every variable.

(c) Variable A is already assigned and constraint propagation has been done. **Circle** all unassigned variables that might be selected by the *Minimum-Remaining-Values* (MRV) heuristic.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	R G B	R G B	R G B

Remaining values: B: 2, C: 2, D: 3, E: 3, F: 3 ⇒ MRV may select **B, C**.

(d) Same situation. **Circle** all unassigned variables that might be selected by the *Degree Heuristic*.

Variables	**A**	**B**	**C**	**D**	**E**	**F**
Domains	**R**	G B	G B	R G B	R G B	R G B

Degree = number of constraints with other *unassigned* variables: B: 2, C: 3, D: 4, E: 3, F: 2 ⇒ the degree heuristic may select **D**.

(e) Assume no variable has been assigned yet. Solve the CSP using backtracking with *forward checking*, choosing variables with MRV, then the degree heuristic (DH), and finally alphabetical order; try values in the order R, G, B. For every step give the MRV value of every region not yet coloured, and the DH values of the regions tied on the minimum MRV.

Step 1

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	3	3	3	3	3	3
DH	2	3	4	4	3	2

Assign to the variable **C** with the value **R** (tie on MRV and DH, broken alphabetically). Domains before this step: A=RGB, B=RGB, C=RGB, D=RGB, E=RGB, F=RGB.
Step 2

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	2	–	2	2	3
DH	1	2		3	2	

Assign to the variable **D** with the value **G** (tie on MRV broken by DH). Domains before this step: A=GB, B=GB, C=R, D=GB, E=GB, F=RGB.
Step 3

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	2	1	–	–	1	2
DH		1			1	

Assign to the variable **B** with the value **B** (tie on MRV and DH, broken alphabetically). Domains before this step: A=GB, B=B, C=R, D=G, E=B, F=RB.
Step 4

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	–	–	–	1	2
DH	0				1	

Assign to the variable **E** with the value **B** (tie on MRV broken by DH). Domains before this step: A=G, B=B, C=R, D=G, E=B, F=RB.
Step 5

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	1	–	–	–	–	1
DH	0					0

Assign to the variable **A** with the value **G** (tie on MRV and DH, broken alphabetically). Domains before this step: A=G, B=B, C=R, D=G, E=B, F=R.
Step 6

Variables	**A**	**B**	**C**	**D**	**E**	**F**
MRV	–	–	–	–	–	1
DH						

Assign to the variable **F** with the value **R** (unique minimum MRV). Domains before this step: A=G, B=B, C=R, D=G, E=B, F=R.
Solution: A=G, B=B, C=R, D=G, E=B, F=R. No backtracking is needed.

Question 8. *Planning a study session.* You are planning a study session with four decisions: the

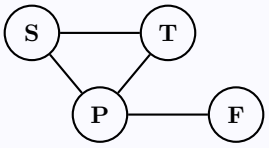
(S)ubject, the (P)lace, the (T)ime and the (F)ood. The candidates are S ∈ {*ai, math, phys*}, P ∈ {*library, cafe, home*}, T ∈ {*morning, afternoon, evening*}, F ∈ {*pizza, salad, noodles*}, with the constraints: (1) the library is open only in the morning or afternoon; (2) math must be studied in the morning; (3) pizza is not allowed in the library; (4) AI cannot be studied at the cafe (no sockets).

(a) Formulate the problem as a CSP: variables, domains and binary constraints.

Variables	**S**	**P**	**T**	**F**
Domains	ai, math, phys	library, cafe, home	morning, afternoon, evening	pizza, salad, noodles

Binary constraints: (1) between P and T, (2) between S and T, (3) between P and F, (4) between S and P.

(b) Draw the constraint graph.



(c) You decide to study in the *evening*. Cross out the eliminated values to show the domains after arc consistency has been enforced.

Variables	**S**	**P**	**T**	**F**
Domains	ai ~~math~~ phys	~~library~~ cafe home	~~morning~~ ~~afternoon~~ **evening**	pizza salad noodles

(1) removes *library*; (2) removes *math*; then no other arc can remove anything: *ai* still has support *home*, and *pizza* has support *cafe*/*home*.

(d) Give one solution, or state that none exists.

For example S=ai, P=home, T=evening, F=pizza (there are 9 solutions with T=evening).

Question 9. Answer the following questions briefly.

(a) Russell & Norvig organise definitions of AI into four categories. Name them, state which one this course adopts and give two reasons.

Thinking humanly (cognitive modelling), thinking rationally (laws of thought), acting humanly (Turing test), acting rationally (**rational agent** — adopted). Reasons: it is more general than the laws-of-thought approach (correct inference is only one way to act rationally; sometimes something must be done with no provably correct choice), and rationality is mathematically well defined, so it is more amenable to scientific development than approaches based on human behaviour or thought.

(b) Explain the difference between the state space and the search tree, and between tree-search and graph-search algorithms.

The state space is the graph of world states and actions (each state once). The search tree contains paths from the initial state; a state may appear in many nodes. Tree-search does not check redundant paths; graph-search keeps a *reached* table and only the best path to each state. Tree-search suits formulations where redundant paths are rare, or when memory is too small for the reached table.

(c) For a uniform tree with branching factor $b = 5$ and the shallowest goal at depth $d = 4$, how many nodes does BFS generate in the worst case? Why is memory a bigger problem than time for BFS?

$b + b^2 + b^3 + b^4 = 5 + 25 + 125 + 625 =$ **780**. All generated nodes stay in memory ($O(b^d)$ space as well as time), and memory is exhausted long before the time limit (lecture table: terabytes needed at depth 10–12).

(d) Name three landscape features on which hill climbing gets stuck. If one run succeeds with probability $p = 0.14$, how many restarts does random-restart hill climbing need on average?

Local maxima, ridges, plateaux (flat local maxima / shoulders). Expected restarts $1/p \approx$ **7**.

(e) Why should variable selection in backtracking be fail-first while value selection is fail-last?

All variables must be assigned anyway, so a doomed branch should fail as early as possible (MRV/degree prune big subtrees). For values we need only one solution, so we try the value most likely to succeed (LCV keeps the most options for the neighbours).

Question 10. True or false? For each statement, circle **T** or **F** and justify your answer in one or two sentences (an answer without a correct justification earns no credit).

1. BFS is cost-optimal whenever all action costs are strictly positive. **T** **F**

FALSE. BFS returns the shallowest goal; it is optimal only when all step costs are equal (e.g. $S \to G$ costing 10 vs $S \to A \to G$ costing 2).

2. Uniform-cost search applies the goal test when a node is generated. **T** **F**

FALSE. It tests when the node is removed from the frontier; testing at generation could return a more expensive path.

3. Every consistent heuristic is admissible. **T** **F**

TRUE. Summing $h(n_i) \le c(n_i, n_{i+1}) + h(n_{i+1})$ along an optimal path gives $h(n) \le h^*(n)$ since $h(\text{goal}) = 0$.

4. If h_1 and h_2 are admissible, then $\max\{h_1, h_2\}$ is admissible and dominates both. **T** **F**

TRUE. At each node it equals one of two lower bounds, and it is $\ge$ each of them by definition.

5. Greedy best-first (graph) search is complete in infinite state spaces. **T** **F**

FALSE. Only in finite spaces; in an infinite space it can follow an infinite path along which h keeps decreasing.

6. A poor move ordering may make alpha-beta return a different root value than minimax. **T** **F**

FALSE. Alpha-beta prunes only branches that cannot affect the decision; ordering changes only how much is pruned.

7. With perfect move ordering alpha-beta examines $O(b^{m/2})$ nodes. **T** **F**

TRUE. The effective branching factor becomes $\sqrt{b}$, so it can search about twice as deep as minimax in the same time.

8. In MCTS with UCB1, once a node has children, each of them is selected at least once before any of them is selected a second time. **T** **F**

TRUE. An unvisited child has UCB1 $= \infty$.

9. Forward checking detects every inconsistency that arc consistency detects. **T** **F**

FALSE. FC only prunes neighbours of the variable just assigned; it misses conflicts between two unassigned variables, which AC propagates.

10. In simulated annealing a worse move ($\Delta E < 0$) is accepted with probability $e^{\Delta E/T}$, which decreases as T decreases. **T** **F**

TRUE. E.g. $\Delta E = -2$: $T = 4 \Rightarrow 0.61$, $T = 1 \Rightarrow 0.14$.